MIMO

SELF-SERVICE PRINTING · MIMO 2.0 · SV-002

Production Lag Forensic Investigation Report

Raspberry Pi, kiosk and print-pipeline investigation of the MIMO 2.0 kiosk (SV-002)

Machine	MIMO 2.0 / SV-002
Target	Raspberry Pi / kiosk / print pipeline
Investigation type	Production forensic investigation
Approach	Read-only production investigation
Classification	Internal · Technical and management audience

Prepared from the supplied production forensic investigation content.

Table of Contents

1 Executive Summary 3

2 Non-Technical Founder Summary 4

3 Exact Root Cause 5

4 Where and How the Lag Happens 6

5 Layer-by-Layer Technical Analysis 7

5.1 Frontend (Chromium / kiosk UI) · 5.2 Vercel deployment · 5.3 Backend / API 7

5.4 Firebase / Storage · 5.5 Raspberry Pi 7

5.6 Network · 5.7 Tailscale · 5.8 CUPS / printer · 5.9 Print progress 9

6 H1–H11 Verdict Table 11

7 Key Evidence & Measurements 12

8 Root-Cause Flow 13

9 Recommended Fixes (Priority Order) 14

10 Final Technical Conclusion 15

11 Final Management Conclusion 15

Machine	MIMO 2.0, kiosk ID SV-002
Target	Raspberry Pi / kiosk / print pipeline
Approach	Read-only production forensic investigation
Objective	Identify the exact source of the observed kiosk and printing lag and distinguish primary causes from contributing factors

This report was compiled solely from the investigation content supplied by the requester. No findings were added or altered.

1 Executive Summary

The investigation found that the **primary bottleneck is the Raspberry Pi running MIMO 2.0 (SV-002)**, specifically the **print-listener / prefetch pipeline**.

The listener's prefetch implementation launches a separate background thread for every relevant paid/pending job **without a concurrency limit**. During a listener startup/reconnect snapshot, this resulted in **125 prefetch threads launched simultaneously**. These concurrent downloads created substantial resource pressure on the Raspberry Pi:

Resource	Observed
Total RAM	~991 MiB
RAM used	~797 MiB
Swap used	~900 MiB of 990 MiB, with active swap/page activity
Prefetch cache	~141 MiB across 204 files in `/tmp/mimo_pre_fetch`
Chromium	~64% combined CPU across key processes
Download problems	27 download timeouts, 10 `RemoteDisconnected` events, 1 `BrokenPipe`
Worst single download	a 62 KB file took approximately 242 seconds

> **Resulting chain**
>
> Unbounded prefetch → many simultaneous downloads → network/socket contention → download failures/retries → temporary cache growth → RAM pressure → swap/I/O pressure → competition with Chromium/listener → delayed document preparation → late print handoff → user-visible lag.

Contributing or separate issues

- Chromium consumes substantial CPU on the Pi.
- Wi-Fi signal is strong, but there is evidence of transmission contention.
- Backend/API operations have a persistent cross-region latency cost.
- Frontend polling and configuration refetches add unnecessary work.
- The SV-002 kiosk is running a stale/mismatched Vercel deployment.
- The SV-002 UI expects `sheetsCompleted`, while the listener does not write that field, which can make the printing UI appear stuck.

These are not equivalent causes. **The strongest evidence points to the Pi-side listener/prefetch workload as the primary production bottleneck.**

2 Non-Technical Founder Summary

What is actually happening?

The printer itself is not the main problem. MIMO 2.0 has a small Raspberry Pi computer that controls the printing process, and that computer is being asked to do too much work at the same time.

When jobs are detected, the software starts downloading files in the background. Instead of limiting how many downloads happen simultaneously, it can start a very large number of downloads at once. During the investigation, **125 downloads were triggered simultaneously**.

The Raspberry Pi has limited memory. Once the machine becomes heavily loaded, it starts using slower storage as additional memory, which makes the entire machine slower. At the same time, the kiosk's Chrome/Chromium browser is also consuming a large amount of CPU.

> **Too many files are being downloaded simultaneously + limited Raspberry Pi resources + heavy browser workload = the whole MIMO 2.0 kiosk becomes slow.**

The internet signal itself is strong. The printer was also not the bottleneck during the investigated lag periods.

What should fix it?

1. Limit the number of simultaneous file downloads.
2. Prevent large download bursts when the listener starts/reconnects.
3. Safely remove unused temporary files.
4. Reduce unnecessary Chromium/browser workload.
5. Fix the separate stale kiosk deployment.
6. Fix the printing progress reporting.
7. Test the complete flow again under simultaneous jobs.

3 Exact Root Cause

> **Primary root cause**
>
> An overloaded SV-002 Raspberry Pi print-listener/prefetch pipeline caused by **unbounded concurrent prefetch downloads** and **RAM-backed cache pressure**.

The listener source code was directly inspected. The prefetch snapshot handler effectively does:

```
for every relevant Firestore change:
    if job is paid/pending:
        start a new background thread
```

There is no semaphore, bounded worker pool, maximum concurrent download count or queue limit. During startup/reconnect, the snapshot produced **125 simultaneous prefetch threads**. This is one of the strongest pieces of evidence in the investigation.

The issue is therefore not simply "the internet is slow". The issue is that **the Pi is generating an excessive number of simultaneous network operations itself.**

Behaviour of the pipeline

- Firestore snapshot → paid/pending jobs detected → one background prefetch thread per job → **no concurrency limit** → large simultaneous download burst.
- The burst causes network/socket contention, timeouts and retries, cache and RAM pressure, then swap/I/O pressure, competition with Chromium and the listener, delayed document preparation, and a late print start.

4 Where and How the Lag Happens

The lag happens **upstream of the printer**, inside the Pi, before the print job reaches CUPS. **Zero CUPS print jobs** were observed during the investigated lag windows, so the observed lag cannot be primarily explained as "the printer is taking too long" or "CUPS is stuck".

Stage	What happens
1. Listener snapshot	Paid/pending jobs are detected on startup/reconnect.
2. Prefetch burst	One background thread per job, with no limit: 125 simultaneous threads.
3. Downloads	Many concurrent Storage downloads cause network/socket contention, timeouts, `RemoteDisconnected` events and retries.
4. Resource pressure	Prefetch cache grows, RAM pressure builds, swap and I/O pressure follow.
5. Competition	Chromium (~64% CPU) and the listener compete for the Pi's limited resources.
6. Preparation	Document preparation is slow, so the job reaches CUPS late.
7. User impact	The user sees "warming up" / printing lag.

> **Separate chain (UI progress only)**
>
> Stale SV-002 Vercel deployment + missing `sheetsCompleted` updates → UI progress appears stuck or outdated. This chain is separate from the primary Pi overload.

The full visual flow is shown in Section 8.

5 Layer-by-Layer Technical Analysis

5.1 Frontend (Chromium / kiosk UI)

- Chromium was a major resource consumer: key processes used approximately **54.5% and 18.2% CPU, about 64% combined**. Chromium is therefore a confirmed resource contributor.
- Chromium should not be described as the primary cause of the prefetch storm. The listener creates excessive work and Chromium competes with that workload for the Pi's limited resources.
- Frontend polling and configuration refetches (for example a configuration request for every digit keypress) add unnecessary work.

5.2 Vercel Deployment

- SV-002 was serving a **stale/mismatched Vercel deployment and an older JavaScript bundle**, so the kiosk was not necessarily running the same frontend build expected from the current repository.
- This is a real production issue and must be fixed. It is **not** the root cause of the Pi's prefetch storm; it explains a different symptom: the UI can appear stuck or show outdated progress behaviour.

5.3 Backend / API

Services are located in different regions, which creates a persistent cross-region latency floor.

Service	Region
API	us-central1
Firestore	asia-south1
Storage	us-east1

Observed timings: `/kiosk/job-status` ≈ **0.57 s**; `/kiosk/print` ≈ **1.15–1.18 s**. This definitely contributes to overall response time. No excessive Firestore/API request storm was found: **backend latency is contributing; excessive API calls are falsified as the primary cause.**

5.4 Firebase / Storage

Firebase/Storage is part of the document download path. The investigation did not establish a global Firebase or Storage outage. The evidence indicates that the Pi's prefetch behaviour creates excessive concurrent download activity: **Firebase/Storage is part of the overloaded path, but the Pi is the component generating the problematic workload.**

5.5 Raspberry Pi

Metric	Observed
Total RAM	~991 MiB
RAM used	~797 MiB
RAM free	~120 MiB
RAM available	~193 MiB
Swap total	~990 MiB
Swap used	~900 MiB
/tmp (tmpfs)	~496 MiB
Prefetch cache	~141 MiB
Prefetch files	204
Throttle status / temperature	0x0 / approximately 49.9 °C

The system also showed **active paging/swap activity**, meaning the machine was operating under significant memory pressure.

Prefetch concurrency

The prefetch snapshot handler starts a new background thread for each relevant paid/pending Firestore change. There is no semaphore, bounded worker pool, maximum concurrent download count or queue limit. During startup/reconnect this produced 125 simultaneous prefetch threads.

Download failure evidence

Observation	Count / value
Prefetches in the same second	125
Download starts	179
Completions	202
Download-attempt failures	29
Timeouts	27
`RemoteDisconnected` events	10
`BrokenPipe`	1
Slowest example	a ~62 KB file took ~242 seconds

During the same period a heartbeat request failed with HTTP 504. This is consistent with a heavily contended Pi/network workload rather than a normal healthy download path.

Cache lifecycle

It is not correct to say that every prefetched file remains permanently. Files that actually reach `process_job` are tracked and deleted after processing. However, the prefetch cache has no independent mechanism to evict files that were prefetched but never consumed by `process_job`: no TTL, no age-based eviction, no size quota and no LRU cleanup. Such files can accumulate and contribute to resource pressure.

5.6 Network

Observation	Value
Signal	approximately –24 dBm
Link quality	70/70
Bit rate	approximately 72.2 Mb/s
RX errors / drops	0
Excessive TX retries	181

There is no evidence of weak Wi-Fi signal being the primary cause. The 181 excessive TX retries support network contention during the heavy prefetch activity. **Network is a contributing factor, not the primary root cause**; the more accurate explanation is that the Pi's excessive simultaneous downloads create the contention.

5.7 Tailscale

There is no evidence from the investigation that Tailscale is responsible for the observed lag, and it should not be blamed as the root cause. **Verdict: not shown as a cause.** The production evidence points toward the local Pi workload and prefetch behaviour instead.

5.8 CUPS / Printer

- CUPS was investigated during the key lag windows. The printers were idle and enabled.
- **Zero CUPS print jobs** were observed during the investigated lag windows, so the lag was happening **before** the print job reached CUPS.
- There are separate printer timeout and toner-related incidents in production, but they do not explain the Pi-side prefetch/resource-exhaustion mechanism established in this investigation.

Verdict: CUPS/printer delay is falsified for the investigated lag windows.

5.9 Print Progress (`sheetsCompleted`)

The SV-002 printing UI expects progress information involving `sheetsCompleted`, but the SV-002 listener does not write this field. The UI can therefore remain at "Warming up printer..." or show very low progress even though other parts of the job lifecycle are progressing, making the user experience appear worse than the actual pipeline state. **Progress reporting is a contributing / separate issue**; it is not the cause of the 125-thread prefetch storm.

6 H1–H11 Verdict Table

ID	Hypothesis	Verdict	Basis
H1	Pi CPU / RAM / I/O contention	CONFIRMED	~797 MiB of ~991 MiB RAM used; ~900 MiB of ~990 MiB swap used; active paging
H2	Thermal / undervoltage throttling	FALSIFIED	Throttle status 0x0; temperature ~49.9 °C
H3	Cache / tmp / disk pressure	CONFIRMED	~141 MiB in 204 prefetch files on a ~496 MiB tmpfs; no eviction for unconsumed files
H4	Excessive Firestore / API calls	FALSIFIED	No Firestore/API request storm found as the primary cause
H5	Backend latency	CONTRIBUTING	Cross-region latency floor (~0.57 s status, ~1.15–1.18 s print)
H6	Frontend re-render / refetch	CONTRIBUTING	Polling and configuration refetches (e.g. per digit keypress) add unnecessary work
H7	Chromium / GPU resource pressure	CONFIRMED	~54.5% + ~18.2% ≈ 64% combined CPU across key Chromium processes
H8	Print pipeline CPU / I/O	CONFIRMED	125 concurrent prefetch threads, timeouts and retries, delayed document preparation
H9	Printer / CUPS completion	FALSIFIED for investigated windows	Zero CUPS jobs during the lag windows; printers idle and enabled
H10	Network instability / contention	CONTRIBUTING	Strong signal and zero RX errors, but 181 excessive TX retries under load
H11	Deployed build mismatch	CONFIRMED	SV-002 served a stale Vercel deployment and older JavaScript bundle

What is NOT the main cause

Candidate	Why it is not the main cause
The physical printer	Not during the investigated lag windows.
CUPS	No CUPS jobs were present during the key lag periods.
Weak Wi-Fi signal	Signal strength was excellent.
Tailscale	No evidence implicated it.
Firebase being globally down	No evidence of a global Firebase/Storage outage.
Excessive API calls	This hypothesis was falsified.
Thermal throttling	Throttle status 0x0, temperature ~49.9 °C.

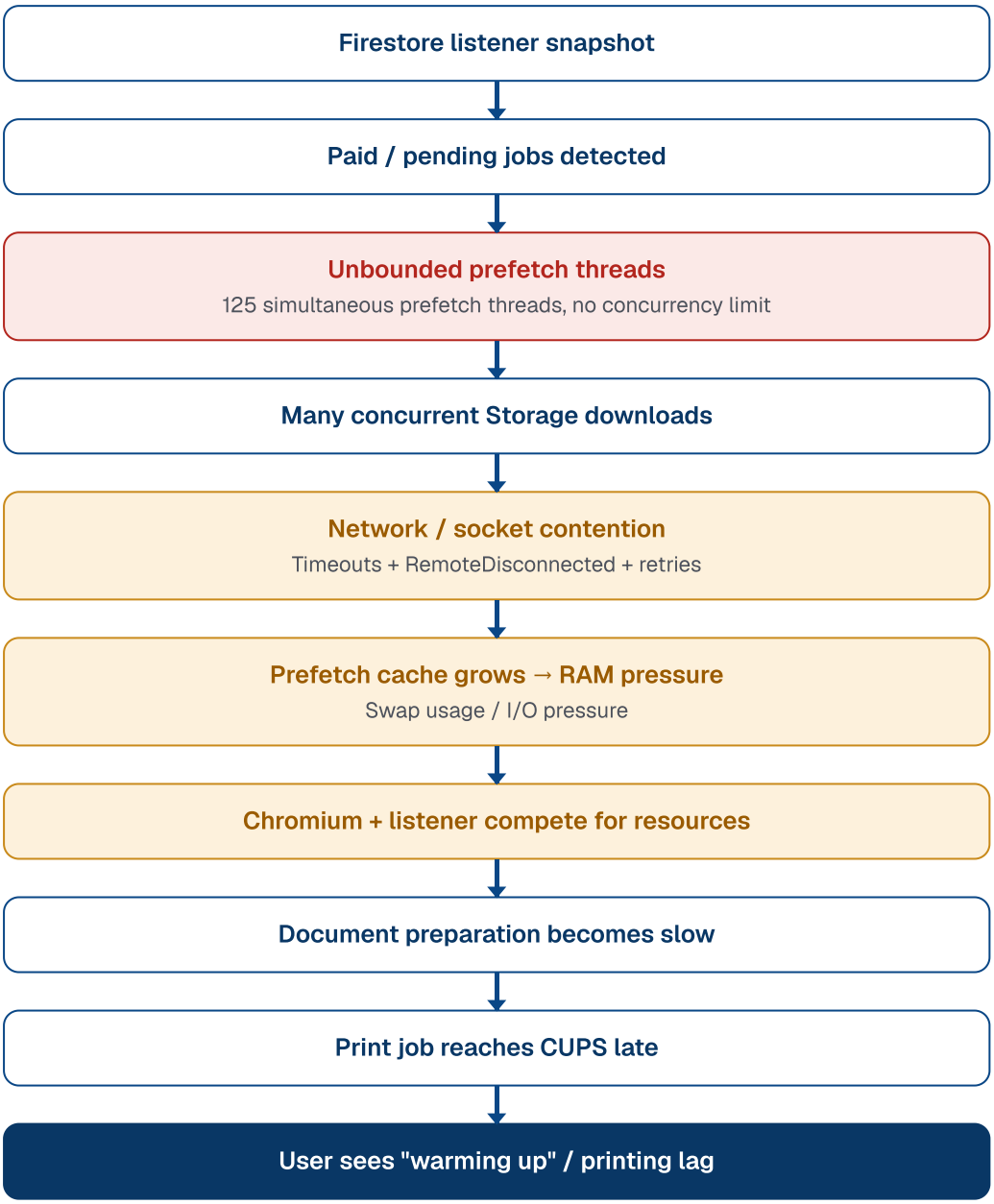
Candidate	Why it is not the main cause
Vercel as the cause of the Pi overload	Vercel has a real separate issue, but it does not create the Pi's prefetch storm.

7 Key Evidence & Measurements

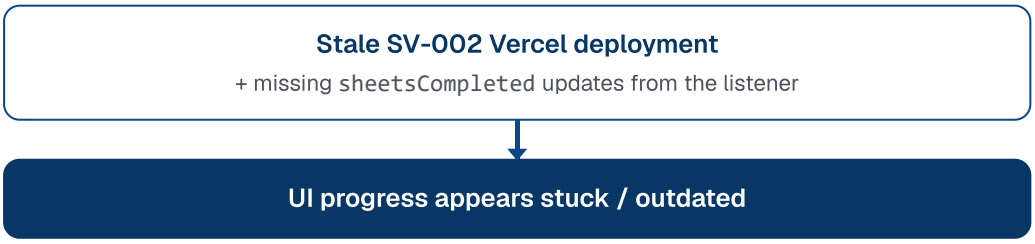
Area	Measurement
Prefetch concurrency	**125** simultaneous threads; no semaphore / worker pool / queue / limit
Downloads	179 starts, 202 completions, 29 failed attempts
Failures	27 timeouts, 10 `RemoteDisconnected`, 1 `BrokenPipe`
Slowest download	~62 KB file in **~242 s**; heartbeat HTTP 504 during the same period
RAM	~797 MiB used of ~991 MiB (≈120 MiB free, ≈193 MiB available)
Swap	~900 MiB used of ~990 MiB, active paging
Cache	~141 MiB, 204 files in `/tmp/mimo_pre_fetch`; /tmp tmpfs ~496 MiB
Chromium	~54.5% + ~18.2% CPU ≈ **64%**
Wi-Fi	−24 dBm, link quality 70/70, 72.2 Mb/s, 0 RX errors/drops, **181 excessive TX retries**
Thermal	Throttle flags 0x0, ~49.9 °C
CUPS	**0** print jobs during the lag windows; printers idle and enabled
Backend timings	`/kiosk/job-status` ≈ 0.57 s; `/kiosk/print` ≈ 1.15–1.18 s
Regions	API us-central1, Firestore asia-south1, Storage us-east1

8 Root-Cause Flow

Primary chain: Raspberry Pi overload



Separate chain: UI progress



9 Recommended Fixes (Priority Order)

#	Fix	Detail
1	**Bound prefetch concurrency**	Do not let every Firestore snapshot change create an unrestricted background thread. Use a bounded worker pool, semaphore or controlled queue, with the maximum number of simultaneous downloads determined through controlled testing rather than guessed. **Most important fix.**
2	**Prevent startup/reconnect storms**	A listener reconnect should not attempt to prefetch hundreds of jobs. Limit prefetching to relevant active, recent jobs likely to be printed; avoid replaying a large historical snapshot as a massive download burst.
3	**Add safe cache eviction**	Maximum cache size, maximum cache age, safe orphan cleanup, and protection for files currently being processed.
4	**Review tmpfs usage**	Evaluate whether the prefetch cache should remain RAM-backed. A disk-backed cache may reduce RAM pressure but must be tested carefully because Raspberry Pi storage has performance and endurance considerations.
5	**Make the listener self-healing**	Detect when the realtime Firestore listener stream becomes unhealthy and reconnect safely. Fallback polling should not be the primary recovery mechanism.
6	**Reduce Chromium workload**	Investigate unnecessary rendering, background work, polling frequency, configuration refetches and component re-renders. Avoid a configuration request for every digit keypress.
7	**Fix SV-002 deployment mismatch**	Ensure SV-002 runs the intended current frontend build, and verify that the actual served bundle equals the expected production build. Do not assume Vercel deployment success means the kiosk is serving that deployment.
8	**Fix progress reporting**	Ensure the fields the kiosk expects, particularly `sheetsCompleted`, are actually written, with one authoritative definition of print progress.
9	**Review backend regions**	Consider reducing cross-region latency between API, Firestore and Storage. This is an optimization/contributor, not the first fix.
10	**Controlled production testing**	After the fixes, test: one job; multiple simultaneous jobs; listener restart; listener reconnect; large documents; multiple files; B&W; colour; network contention; cache growth; RAM; swap; CPU; CUPS handoff; actual printer completion. Measure before and after.

Do not use these as permanent fixes

These may temporarily change symptoms but do not solve the root cause: manually deleting `/tmp/mimo_pre_fetch`; restarting the Raspberry Pi; restarting Chromium; restarting CUPS; restarting Tailscale; increasing polling frequency; or simply increasing Pi resources without fixing the download storm. The software workload must be controlled first.

10 Final Technical Conclusion

The primary production bottleneck on SV-002 is an **unbounded prefetch workload in the Pi listener**. It launched **125 simultaneous prefetch threads** from one snapshot, causing download timeouts and disconnects, cache growth, RAM exhaustion, and swap/I/O pressure that competed with Chromium and the listener. This delayed document preparation before the job reached CUPS, and zero CUPS jobs were present during the lag windows.

Verdict	Hypotheses
CONFIRMED	H1, H3, H7, H8, H11
CONTRIBUTING	H5, H6, H10
FALSIFIED	H2, H4, H9 (for the investigated windows)

The stale Vercel deployment and the missing `sheetsCompleted` writes are real but separate issues. They affect UI progress, not the Pi overload.

11 Final Management Conclusion

MIMO 2.0 feels slow because its Raspberry Pi is overloaded by a software behaviour: it starts too many file downloads at the same moment and keeps unused temporary files, which exhausts the small machine's memory while the kiosk browser also uses a large share of its processor.

The printer, the Wi-Fi signal, Tailscale and Firebase are not the main cause.

> **What to do**
>
> Fix the Pi first: limit simultaneous downloads and clean up the cache. Two further fixes should follow: update the kiosk's outdated website deployment, and repair the progress display, which currently can look stuck. Then retest under realistic multi-job conditions.